

所以對你來說，比較合理的是：

Weighted sampling 或 loss reweighting

而不是 full oversampling。

這也剛好回到你問的「改 source」。

2. 如何改 Ultralytics source ?

我會推薦這條流程。

Step 1 : clone repo

```
git clone https://github.com/ultralytics/ultralytics.git
cd ultralytics
```

官方 repo :

[Ultralytics GitHub Repo ↗](#)

Step 2 : editable install

在 virtual env 裡：

```
pip uninstall ultralytics
pip install -e .
```

這很重要。

-e = editable mode

意思是：

你改 source code 後：

不用重新 install。

確認是不是用 local repo

Python :

```
import ultralytics  
print(ultralytics.__file__)
```

應該看到 :

```
/path/to/your/ultralytics/ultralytics/__init__.py
```



不是 :

```
site-packages
```



Step 3 : 找 classification trainer

通常看 :

```
ultralytics/models/yolo/classify/
```



你大概會看到 :

```
train.py  
val.py  
predict.py
```



我最關注 :

```
ultralytics/models/yolo/classify/train.py
```



你要改哪裡 ?

取決於方法。

Method A : 改 Loss (我最推薦你先做)

這是 :

Focal Loss + class weights

比改 dataloader 簡單。

找 criterion

在 classification trainer 裡找：

```
CrossEntropyLoss
```

或：

```
self.loss
```

你可能會看到類似：

```
loss = torch.nn.functional.cross_entropy(  
    preds,  
    batch["cls"]  
)
```

(版本不同可能位置不同)

加入 Focal Loss

新增：

```
import torch  
import torch.nn as nn  
import torch.nn.functional as F
```

加 class weight

假設你的 36 classes count：

```
class_counts = [  
    100000, # normal  
    800,  
    1200,  
    ...  
]
```

算 weights :

$$w_c = \frac{\sum_i n_i}{n_c}$$

```
counts = torch.tensor(class_counts).float()  
  
weights = counts.sum() / counts  
weights = weights / weights.mean()
```

Focal Loss class

```
class FocalLoss(nn.Module):  
    def __init__(self, alpha=None, gamma=2):  
        super().__init__()  
        self.alpha = alpha  
        self.gamma = gamma  
  
    def forward(self, logits, targets):  
        ce = F.cross_entropy(  
            logits,  
            targets,  
            reduction='none',  
            weight=self.alpha.to(logits.device)  
        )  
  
        pt = torch.exp(-ce)  
        loss = ((1 - pt) ** self.gamma) * ce  
        return loss.mean()
```

初始化：

```
criterion = FocalLoss(  
    alpha=weights,  
    gamma=2  
)
```

training：

原本：

```
loss = cross_entropy(preds, labels)
```

改：

```
loss = criterion(preds, labels)
```

Method B：改 Weighted Sampler

這要改 dataloader。

稍微麻煩。

找 dataloader build

找：

```
build_dataloader
```

或：

```
DataLoader(
```

通常在：

```
ultralytics/data/
```



你可能看到：

```
loader = DataLoader(  
    dataset,  
    batch_size=batch_size,  
    shuffle=True,  
    num_workers=workers  
)
```

改成：

```
from torch.utils.data import WeightedRandomSampler
```

假設 dataset 有：

```
dataset.samples
```

每個 sample：

```
(path, class_id)
```

算 weight：

```
from collections import Counter

targets = [cls for _, cls in dataset.samples]

class_counts = Counter(targets)

sample_weights = [
    1.0 / class_counts[t]
    for t in targets
]
```

sampler :

```
sampler = WeightedRandomSampler(
    sample_weights,
    num_samples=len(sample_weights),
    replacement=True
)
```

DataLoader :

```
loader = DataLoader(
    dataset,
    batch_size=batch_size,
    sampler=sampler,
    shuffle=False,
    num_workers=workers
)
```

但我其實更推薦第三種（你還沒提到）

老實說，根據你的 case：

- 100:1 imbalance
- company environment
- 不想 dataset 爆炸

- 用 YOLO cls

我最推薦的其實是：

Downsample normal + Focal Loss

而不是：

- pure oversampling
- pure weighted sampler

例如：

```
normal = 100000 → 每個 epoch 隨機取 20000  
defect avg = 1000
```



變：

20:1

而不是：

100:1

原因：

對晶片 normal 圖：

很多高度 redundant。

不像 defect 那麼珍貴。

我的最終建議排序（更新版）

第一推薦

Focal Loss + class weights

- 改 source 中等
 - 效果大
-

第二推薦

Downsample normal

- 工程簡單